Policy Gradient Methods

Alvaro Soto

Computer Science Department (DCC), PUC

DQN requires a large replay buffer

- Fills buffer *offline* before learning $\Rightarrow$ costly in slow/expensive environments

DQN requires a large replay buffer

- Fills buffer *offline* before learning $\Rightarrow$ costly in slow/expensive environments
- Training/deployment bottleneck: robotics, clinical trials, etc.

DQN requires a large replay buffer

- Fills buffer *offline* before learning $\Rightarrow$ costly in slow/expensive environments
- Training/deployment bottleneck: robotics, clinical trials, etc.

DQN requires a large replay buffer

- Fills buffer *offline* before learning $\Rightarrow$ costly in slow/expensive environments
- Training/deployment bottleneck: robotics, clinical trials, etc.

Q-function is only a proxy

- We want π^* but we learn Q^* first, two main inconveniences:

DQN requires a large replay buffer

- Fills buffer *offline* before learning $\Rightarrow$ costly in slow/expensive environments
- Training/deployment bottleneck: robotics, clinical trials, etc.

Q-function is only a proxy

- We want π^* but we learn Q^* first, two main inconveniences:
  - Errors in Q propagate to π via $\arg\max$

DQN requires a large replay buffer

- Fills buffer *offline* before learning $\Rightarrow$ costly in slow/expensive environments
- Training/deployment bottleneck: robotics, clinical trials, etc.

Q-function is only a proxy

- We want π^* but we learn Q^* first, two main inconveniences:
  - Errors in Q propagate to π via $\arg\max$
  - In the discrete case, $\arg\max_a Q(s, a)$ is not a main complication, however, in continuous action spaces (ex. robotics apps), this requires a costly optimization.

Learning a policy directly: Planning to act in the world

Objective Function:

Maximize expected cumulative and discounted reward

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_t \gamma^t \; r_t\right]$$

Learning a policy directly: Planning to act in the world

Objective Function:

Maximize expected cumulative and discounted reward

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_t \gamma^t \; r_t\right]$$

This is equivalent to optimize the value function averaged over the start-state distribution: $J(\theta) = \mathbb{E}_{s_0}[V^{\pi_\theta}(s_0)]$

Learning a policy directly: Planning to act in the world

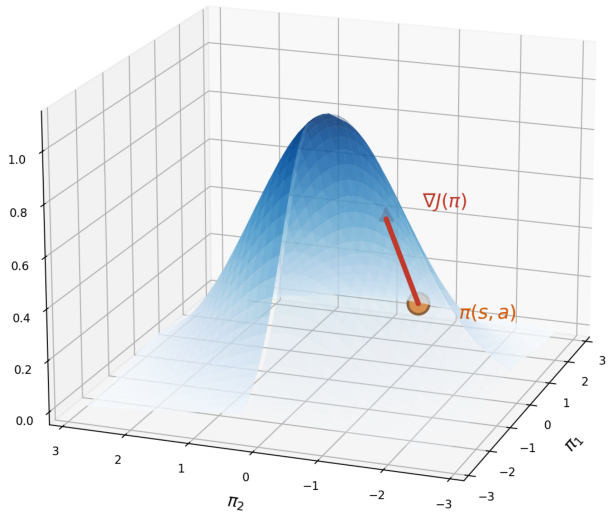
Objective Function:

Maximize expected cumulative and discounted reward

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_t \gamma^t \, r_t\right]$$

This is equivalent to optimize the value function averaged over the start-state distribution: $J(\theta) = \mathbb{E}_{s_0}[V^{\pi_\theta}(s_0)]$

We will optimize the underlying policy using gradient ascent



Learning a policy directly: Planning to act in the world

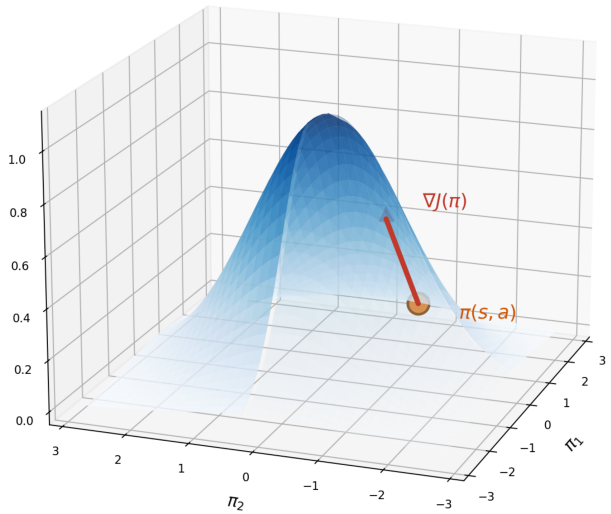
Objective Function:

Maximize expected cumulative and discounted reward

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_t \gamma^t \, r_t\right]$$

This is equivalent to optimize the value function averaged over the start-state distribution: $J(\theta) = \mathbb{E}_{s_0}[V^{\pi_\theta}(s_0)]$

We will optimize the underlying policy using gradient ascent



Key question: how do we compute $\nabla_\theta J(\theta)$?

(the environment is not differentiable!)

Policy: $\pi_\theta(a|s)$ (e.g. a neural network with weights θ)

Data given by episodes or trajectories: $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_T)$

Reward of a trajectory:

$$G(\tau) = \sum_{t=0}^{T-1} \gamma^t \, r_t$$

Observation: sum goes to $T-1$ because in our notation S_T is the terminal state.

Policy: $\pi_\theta(a|s)$ (e.g. a neural network with weights θ)

Data given by episodes or trajectories: $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_T)$

Reward of a trajectory:

$$G(\tau) = \sum_{t=0}^{T-1} \gamma^t \, r_t$$

Observation: sum goes to $T-1$ because in our notation S_T is the terminal state.

Objective Function

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)] = \sum_{\tau} P(\tau;\theta)\, G(\tau)$$

$P(\tau;\theta)$: probability of trajectory τ under policy π_θ

Policy: $\pi_\theta(a|s)$ (e.g. a neural network with weights θ)

Data given by episodes or trajectories: $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_T)$

Reward of a trajectory:

$$G(\tau) = \sum_{t=0}^{T-1} \gamma^t \, r_t$$

Observation: sum goes to $T-1$ because in our notation S_T is the terminal state.

Objective Function

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)] = \sum_{\tau} P(\tau;\theta)\, G(\tau)$$

$P(\tau;\theta)$: probability of trajectory τ under policy π_θ

Goal: find $\theta^* = \arg\max_\theta J(\theta)$ using gradient ascent

Gradient ascent: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

Challenge: compute gradient of the objetive function:

$$\nabla_\theta J(\theta) = \nabla_\theta \sum_\tau P(\tau;\theta)\, G(\tau) = \sum_\tau G(\tau)\, \nabla_\theta P(\tau;\theta)$$

Challenge: compute gradient of the objetive function:

$$\nabla_\theta J(\theta) = \nabla_\theta \sum_\tau P(\tau;\theta)\, G(\tau) = \sum_\tau G(\tau)\, \nabla_\theta P(\tau;\theta)$$

As usual in RL, we assume a MDP structure for $P(\tau;\theta)$, where:

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_T)$$

Then:

$$P(\tau;\theta) = p(s_0)\pi_\theta(a_0|s_0)p(s_1|a_0,s_0) \;\; \ldots \;\; \pi_\theta(a_{T-1}|s_{T-1})p(s_T|a_{T-1},s_{T-1})$$

Observation. Notice a key subtlety, we extend classic RL, where the policy is a deterministic function, to a more general case, where the policy is a probability distribution (PDF).

Policy as a Probability Distribution

Classic RL view: the policy is a *deterministic* mapping from states to actions:

$$a_t = \pi(s_t)$$

Policy as a Probability Distribution

Classic RL view: the policy is a *deterministic* mapping from states to actions:

$$a_t = \pi(s_t)$$

Policy gradient view: it uses a more general definition where the policy is a *probability distribution* over actions, conditioned on the state:

$$\pi_\theta(a \mid s) : \mathcal{S} \to \Delta(\mathcal{A})$$

Policy as a Probability Distribution

Classic RL view: the policy is a *deterministic* mapping from states to actions:

$$a_t = \pi(s_t)$$

Policy gradient view: it uses a more general definition where the policy is a *probability distribution* over actions, conditioned on the state:

$$\pi_\theta(a \mid s) : \mathcal{S} \to \Delta(\mathcal{A})$$

Key subtlety: it is a Markov kernel, not one distribution

$\pi_\theta(a \mid s)$ is **not** a single probability distribution. For each fixed state s, $\pi_\theta(\cdot \mid s)$ is a distribution over the action space. The policy is the whole *family* $\{\pi_\theta(\cdot \mid s)\}_{s \in \mathcal{S}}$.

Policy as a Probability Distribution

Classic RL view: the policy is a *deterministic* mapping from states to actions:

$$a_t = \pi(s_t)$$

Policy gradient view: it uses a more general definition where the policy is a *probability distribution* over actions, conditioned on the state:

$$\pi_\theta(a \mid s) : \mathcal{S} \to \Delta(\mathcal{A})$$

Key subtlety: it is a Markov kernel, not one distribution

$\pi_\theta(a \mid s)$ is **not** a single probability distribution. For each fixed state s, $\pi_\theta(\cdot \mid s)$ is a distribution over the action space. The policy is the whole *family* $\{\pi_\theta(\cdot \mid s)\}_{s \in \mathcal{S}}$.

Why this matters:

- Stochasticity gives natural *exploration* (no ϵ-greedy needed).
- $\pi_\theta(a \mid s)$ is differentiable in $\theta \Rightarrow$ enables $\nabla_\theta \log \pi_\theta(a \mid s)$.
- Works for *continuous* action spaces, where $\arg\max_a Q(s, a)$ was the costly bottleneck (slide 2).

Going back to our challenge: $\nabla_\theta J(\theta)$

We need to compute:

$$\nabla_\theta J(\theta) = \nabla_\theta \sum_\tau P(\tau;\theta)\, G(\tau) = \sum_\tau G(\tau)\, \nabla_\theta P(\tau;\theta)$$

where,
$P(\tau;\theta) = p(s_0)\pi_\theta(a_0|s_0)p(s_1|a_0,s_0) \;\; \ldots \;\; \pi_\theta(a_{T-1}|s_{T-1})p(s_T|a_{T-1},s_{T-1})$

Problem: we cannot calculate $\nabla_\theta P(\tau;\theta)$ directly. Transition function $p(s_t|a_{t-1},s_{t-1})$ is unknown

Going back to our challenge: $\nabla_\theta J(\theta)$

We need to compute:

$$\nabla_\theta J(\theta) = \nabla_\theta \sum_\tau P(\tau;\theta)\, G(\tau) = \sum_\tau G(\tau)\, \nabla_\theta P(\tau;\theta)$$

where,
$P(\tau;\theta) = p(s_0)\pi_\theta(a_0|s_0)p(s_1|a_0,s_0) \;\; \ldots \;\; \pi_\theta(a_{T-1}|s_{T-1})p(s_T|a_{T-1},s_{T-1})$

Problem: we cannot calculate $\nabla_\theta P(\tau;\theta)$ directly. Transition function $p(s_t|a_{t-1},s_{t-1})$ is unknown

Key insight: Log-derivative trick

$$\nabla_\theta \log f(x) = \frac{\nabla_\theta f(x)}{f(x)} \quad \Longrightarrow \quad \nabla_\theta f(x) = f(x)\, \nabla_\theta \log f(x)$$

Apply with $f(x) = P(\tau;\theta)$:

$$\nabla_\theta P(\tau;\theta) = P(\tau;\theta)\, \nabla_\theta \log P(\tau;\theta)$$

We need:

$$\nabla_\theta J(\theta) = \sum_\tau G(\tau)\, \nabla_\theta P(\tau;\theta)$$

and

$$\nabla_\theta P(\tau;\theta) = P(\tau;\theta)\, \nabla_\theta \log P(\tau;\theta)$$

We need:

$$\nabla_\theta J(\theta) = \sum_\tau G(\tau)\, \nabla_\theta P(\tau;\theta)$$

and

$$\nabla_\theta P(\tau;\theta) = P(\tau;\theta)\, \nabla_\theta \log P(\tau;\theta)$$

then

$$\nabla_\theta J(\theta) = \sum_\tau G(\tau)\, P(\tau;\theta)\, \nabla_\theta \log P(\tau;\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[G(\tau)\, \nabla_\theta \log P(\tau;\theta)]$$

We need:

$$\nabla_\theta J(\theta) = \sum_\tau G(\tau)\, \nabla_\theta P(\tau;\theta)$$

and

$$\nabla_\theta P(\tau;\theta) = P(\tau;\theta)\, \nabla_\theta \log P(\tau;\theta)$$

then

$$\nabla_\theta J(\theta) = \sum_\tau G(\tau)\, P(\tau;\theta)\, \nabla_\theta \log P(\tau;\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[G(\tau)\, \nabla_\theta \log P(\tau;\theta)]$$

$$P(\tau;\theta) = p(s_0)\pi_\theta(a_0|s_0)p(s_1|a_0,s_0) \;\; \ldots \;\; \pi_\theta(a_{T-1}|s_{T-1})p(s_T|a_{T-1},s_{T-1})$$

$$\log P(\tau;\theta) = \log p(s_0) + \sum_{t=0}^{T-1} \underbrace{\log \pi_\theta(a_t|s_t)}_{\text{depends on } \theta} + \sum_{t=0}^{T-1} \underbrace{\log p(s_{t+1}|s_t,a_t)}_{\text{no depends on } \theta}$$

The environment terms (transition functions) vanish when we take ∇_θ:

$$\nabla_\theta \log P(\tau;\theta) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)$$

Almost there ... putting all together

Our original objetive is : $J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_t \gamma^t \, r_t\right]$

Using as a proxy the expected return over trajectories τ sampled from π_θ:

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[G(\tau)] = \sum_\tau P(\tau;\theta)\, G(\tau)$$

Optimizing using gradient ascent, and applying the gradient trick:

$$\nabla_\theta J(\theta) = \sum_\tau G(\tau)\nabla_\theta P(\tau;\theta) \; = \sum_\tau G(\tau)P(\tau;\theta)\nabla_\theta \log P(\tau;\theta)$$

Appying the vanishing environment result:

$$\nabla_\theta J(\theta) = \sum_\tau G(\tau)P(\tau;\theta) \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)$$

Policy Gradient Theorem:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[G(\tau) \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)\right]$$

Estimating the Gradient using Monte Carlo

The gradient is an expectation, we can use sampling to obtain an estimation $\nabla_\theta \hat{J}(\theta)$ using N episodes $\tau^{(1)}, \ldots, \tau^{(N)}$:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[G(\tau) \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) \right] \approx \frac{1}{N} \sum_{i=1}^{N} \left[G(\tau^{(i)}) \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t^{(i)}|s_t^{(i)}) \right]$$

1. **Collect** N complete episodes $\tau^{(1)}, \ldots, \tau^{(N)}$ under current π_θ
2. **Compute** return $G(\tau^{(i)})$ for each episode
3. **Compute** weighted average of gradient estimates:
   $\nabla_\theta \hat{J}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left[G(\tau^{(i)}) \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t^{(i)}|s_t^{(i)}) \right]$
4. **Update:** $\theta \leftarrow \theta + \alpha \nabla_\theta \hat{J}(\theta)$

REINFORCE Algorithm

We are now ready to present our first working algorithm: REINFORCE (Williams, 1992). It implements the previous ideas with two refinements suited to gradient based optimization:

- Collecting N trajectories for each gradient update can be cumbersome, so it approximates the gradient using a single sampled trajectory per update (N=1).
- It applies a per-step update using the reward-to-go $G_t \leftarrow \sum_{k=t}^{T-1} \gamma^{k-t} r_k$ instead of a single aggregated update for the full-episode.

Algorithm 1 REINFORCE (Williams, 1992)

Initialize policy parameters θ randomly
repeat
  Generate episode $\tau = (s_0, a_0, r_0, \ldots, s_T)$ following π_θ
  for $t = 0, 1, \ldots, T-1$ **do**
    Compute return: $G_t \leftarrow \sum_{k=t}^{T-1} \gamma^{k-t} r_k$
    Update: $\theta \leftarrow \theta + \alpha\, G_t\, \nabla_\theta \log \pi_\theta(a_t|s_t)$
  end for
until converged

OJO: calculation of G_t considers only rewards after t. This is a causality issue, action a_t is only responsible of rewards after t.

REINFORCE Algorithm

We are now ready to present our first working algorithm: REINFORCE (Williams, 1992). It implements the previous ideas with two refinements suited to gradient based optimization:

- Collecting N trajectories for each gradient update can be cumbersome, so it approximates the gradient using a single sampled trajectory per update (N=1).
- It applies a per-step update using the reward-to-go $G_t \leftarrow \sum_{k=t}^{T-1} \gamma^{k-t} r_k$ instead of a single aggregated update for the full-episode.

Algorithm 1 REINFORCE (Williams, 1992)

Initialize policy parameters θ randomly
repeat
  Generate episode $\tau = (s_0, a_0, r_0, \ldots, s_T)$ following π_θ
  for $t = 0, 1, \ldots, T-1$ **do**
    Compute return: $G_t \leftarrow \sum_{k=t}^{T-1} \gamma^{k-t} r_k$
    Update: $\theta \leftarrow \theta + \alpha\, G_t\, \nabla_\theta \log \pi_\theta(a_t|s_t)$
  end for
until converged

OJO: calculation of G_t considers only rewards after t. This is a causality issue, action a_t is only responsible of rewards after t.

Key question: how do we compute $\nabla_\theta \log \pi_\theta(a_t|s_t)$?

REINFORCE Algorithm

We are now ready to present our first working algorithm: REINFORCE (Williams, 1992). It implements the previous ideas with two refinements suited to gradient based optimization:

- Collecting N trajectories for each gradient update can be cumbersome, so it approximates the gradient using a single sampled trajectory per update (N=1).
- It applies a per-step update using the reward-to-go $G_t \leftarrow \sum_{k=t}^{T-1} \gamma^{k-t} r_k$ instead of a single aggregated update for the full-episode.

Algorithm 1 REINFORCE (Williams, 1992)

Initialize policy parameters θ randomly
repeat
  Generate episode $\tau = (s_0, a_0, r_0, \ldots, s_T)$ following π_θ
  for $t = 0, 1, \ldots, T-1$ **do**
    Compute return: $G_t \leftarrow \sum_{k=t}^{T-1} \gamma^{k-t} r_k$
    Update: $\theta \leftarrow \theta + \alpha\, G_t\, \nabla_\theta \log \pi_\theta(a_t|s_t)$
  end for
until converged

OJO: calculation of G_t considers only rewards after t. This is a causality issue, action a_t is only responsible of rewards after t.

Key question: how do we compute $\nabla_\theta \log \pi_\theta(a_t|s_t)$?

It is the **gradient of a policy**. In our case, the **gradient of a NN** ?

Connection to Supervised Learning

Recall: in supervised learning with cross-entropy loss $\mathcal{L}_{\text{SL}}(\theta)$:

$$\mathcal{L}_{\text{SL}}(\theta) = -\sum_i \log \pi_\theta(y_i|x_i) \quad \Rightarrow \quad \nabla_\theta \mathcal{L}_{\text{SL}} = -\sum_i \nabla_\theta \log \pi_\theta(y_i|x_i)$$

REINFORCE gradient (per step):

$$\nabla_\theta J(\theta) \;\propto\; \sum_t G_t \cdot \nabla_\theta \log \pi_\theta(a_t|s_t)$$

Key mapping

Supervised learning		REINFORCE
input x_i	$\longleftrightarrow$	state s_t
label y_i	$\longleftrightarrow$	sampled action a_t
sample weight $= 1$	$\longleftrightarrow$	sample weight $= G_t$

Connection to Supervised Learning

Recall: in supervised learning with cross-entropy loss $\mathcal{L}_{\text{SL}}(\theta)$:

$$\mathcal{L}_{\text{SL}}(\theta) = -\sum_i \log \pi_\theta(y_i|x_i) \quad \Rightarrow \quad \nabla_\theta \mathcal{L}_{\text{SL}} = -\sum_i \nabla_\theta \log \pi_\theta(y_i|x_i)$$

REINFORCE gradient (per step):

$$\nabla_\theta J(\theta) \;\propto\; \sum_t G_t \cdot \nabla_\theta \log \pi_\theta(a_t|s_t)$$

Key mapping

Supervised learning		**REINFORCE**
input x_i	$\longleftrightarrow$	state s_t
label y_i	$\longleftrightarrow$	sampled action a_t
sample weight $= 1$	$\longleftrightarrow$	sample weight $= G_t$

Key Result

Minimizing $\mathcal{L}_{\text{SL}}$ with label a_t and sample weight G_t **is** equivalent to the REINFORCE update

Define the **surrogate loss**:

$$\mathcal{L}_{\text{PG}}(\theta) = -\frac{1}{T}\sum_{t=0}^{T} G_t \cdot \log \pi_\theta(a_t|s_t)$$

Why "surrogate"?
The true objective $J(\theta)$ is not differentiable through the environment.
$\mathcal{L}_{\text{PG}}$ has the **same gradient** as $J(\theta)$ w.r.t. θ, but is differentiable.

Implementation

- Collect episode τ
- Compute returns $\{G_t\}$
- Treat (s_t, a_t) as labeled data
- Minimize $\mathcal{L}_{\text{PG}}$ with Adam/SGD

In PyTorch

```
log_probs =
policy(states).log_prob(actions)
loss = -(returns *
log_probs).mean()
loss.backward()
```

This is really great! we can implement policy gradient optimization as SL classification, we just need to reweight the cross-entropy by G_t.

The weighting factor G_t has an important effect

At each timestep t:

$$\nabla_\theta J(\theta) \propto \mathbb{E}\left[\underbrace{G_t}_{\substack{\text{how good was} \\ \text{the return?}}} \underbrace{\nabla_\theta \log \pi_\theta(a_t|s_t)}_{\substack{\text{direction to make} \\ a_t \text{ more likely in } s_t}} \right]$$

High return $G_t > 0$

Increase the probability of a_t in state s_t

$\Rightarrow$ **"REINFORCE" good actions**

Low (negative) return $G_t < 0$

Decrease the probability of a_t in state s_t

$\Rightarrow$ **Suppress bad actions**

Obs: We will revisit this idea later, but instead of weighting by the absolute value of G_t, we will weight by its *relative* effect: how much better or worse a_t was compared to an average or baseline return at s_t. We will call this relative quantity the **advantage** A_t.

REINFORCE: Limitations

On-policy

- Must collect new data after *every* update
- Old episodes cannot be reused
- Very sample-inefficient

REINFORCE: Limitations

On-policy

- Must collect new data after *every* update
- Old episodes cannot be reused
- Very sample-inefficient

Monte Carlo returns

- Must wait for episode to end
- G_t depends on many future random steps
- High variance gradients

REINFORCE: Limitations

On-policy

- Must collect new data after *every* update
- Old episodes cannot be reused
- Very sample-inefficient

Monte Carlo returns

- Must wait for episode to end
- G_t depends on many future random steps
- High variance gradients

Large updates destabilize policy

- A single gradient step can drastically change π_θ
- New (bad) policy collects worse data (on-policy)
- Recovery is slow: training can collapse

REINFORCE: Limitations

On-policy

- Must collect new data after *every* update
- Old episodes cannot be reused
- Very sample-inefficient

Monte Carlo returns

- Must wait for episode to end
- G_t depends on many future random steps
- High variance gradients

Large updates destabilize policy

- A single gradient step can drastically change π_θ
- New (bad) policy collects worse data (on-policy)
- Recovery is slow: training can collapse

Core question:
Can we reuse data from *old* policies and still optimize the *current* policy safely?

Reusing Old Data: Importance Sampling

Problem: the batch was generated by the old policy $\pi_{\theta_{\text{old}}}$, but we want to improve π_θ

Importance sampling corrects for the distribution mismatch:

$$\mathbb{E}_{\tau\sim\pi_{\theta_{\text{old}}}}\left[\frac{P(\tau;\theta)}{P(\tau;\theta_{\text{old}})}G(\tau)\right] = \mathbb{E}_{\tau\sim\pi_\theta}\left[G(\tau)\right] = J(\theta)$$

Reusing Old Data: Importance Sampling

Problem: the batch was generated by the old policy $\pi_{\theta_{\text{old}}}$, but we want to improve π_θ

Importance sampling corrects for the distribution mismatch:

$$\mathbb{E}_{\tau\sim\pi_{\theta_{\text{old}}}}\left[\frac{P(\tau;\theta)}{P(\tau;\theta_{\text{old}})}G(\tau)\right] = \mathbb{E}_{\tau\sim\pi_\theta}[G(\tau)] = J(\theta)$$

The environment cancels again (same mechanism as slide 8)

$$\frac{P(\tau;\theta)}{P(\tau;\theta_{\text{old}})} = \frac{\cancel{p(s_0)}\prod_t \pi_\theta(a_t|s_t)\,\cancel{p(s_{t+1}|s_t,a_t)}}{\cancel{p(s_0)}\prod_t \pi_{\theta_{\text{old}}}(a_t|s_t)\,\cancel{p(s_{t+1}|s_t,a_t)}} = \prod_{t=0}^{T-1}\underbrace{\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}}_{r_t(\theta)}$$

Reusing Old Data: Importance Sampling

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\prod_{t=0}^{T-1} \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} G(\tau) \right]$$

Reusing Old Data: Importance Sampling

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\prod_{t=0}^{T-1} \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} G(\tau) \right]$$

Problem: This estimator is not practical. Two main problems:

1. A product of T ratios leads to exponential variance grow in T

Reusing Old Data: Importance Sampling

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\prod_{t=0}^{T-1} \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} G(\tau) \right]$$

Problem: This estimator is not practical. Two main problems:

1. A product of T ratios leads to exponential variance grow in T
   - Ex. With $T = 100$, a uniform $r_t = 1.02$ gives weight 7.2, while $r_t = 0.98$ gives 0.13.

Reusing Old Data: Importance Sampling

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\prod_{t=0}^{T-1} \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} G(\tau) \right]$$

Problem: This estimator is not practical. Two main problems:

1. A product of T ratios leads to exponential variance grow in T
   - Ex. With $T = 100$, a uniform $r_t = 1.02$ gives weight 7.2, while $r_t = 0.98$ gives 0.13.
   - A few trajectories take all the weight.

Reusing Old Data: Importance Sampling

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\prod_{t=0}^{T-1} \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} G(\tau) \right]$$

Problem: This estimator is not practical. Two main problems:

1. A product of T ratios leads to exponential variance grow in T
   - Ex. With $T = 100$, a uniform $r_t = 1.02$ gives weight 7.2, while $r_t = 0.98$ gives 0.13.
   - A few trajectories take all the weight.
2. The whole trajectory carries a single return $G(\tau)$: every step is credited identically, whether it helped or hurt

Reusing Old Data: Importance Sampling

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\prod_{t=0}^{T-1} \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} G(\tau) \right]$$

Problem: This estimator is not practical. Two main problems:

1. A product of T ratios leads to exponential variance grow in T
   - Ex. With $T = 100$, a uniform $r_t = 1.02$ gives weight 7.2, while $r_t = 0.98$ gives 0.13.
   - A few trajectories take all the weight.
2. The whole trajectory carries a single return $G(\tau)$: every step is credited identically, whether it helped or hurt
   - Ex. a 100-step episode where the action at t=3 was excellent and the action at t=7 was terrible

Reusing Old Data: Importance Sampling

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\prod_{t=0}^{T-1} \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} G(\tau) \right]$$

Problem: This estimator is not practical. Two main problems:

1. A product of T ratios leads to exponential variance grow in T
   - Ex. With $T = 100$, a uniform $r_t = 1.02$ gives weight 7.2, while $r_t = 0.98$ gives 0.13.
   - A few trajectories take all the weight.
2. The whole trajectory carries a single return $G(\tau)$: every step is credited identically, whether it helped or hurt
   - Ex. a 100-step episode where the action at t=3 was excellent and the action at t=7 was terrible
   - Estimator has no way to say "this step deserves credit, that one deserves blame"

Solving problem 1: one ratio per step instead of a product

Reuse a batch collected with $\pi_{\theta_{\text{old}}}$, but reweight *one transition at a time* instead of whole trajectories:

$$\nabla_\theta J(\theta) \;\approx\; \mathbb{E}_{s_t \sim \rho_{\theta_{\text{old}}},\, a_t \sim \pi_{\theta_{\text{old}}}(\cdot|s_t)} \Big[\, r_t(\theta)\, G_t\, \nabla_\theta \log \pi_\theta(a_t|s_t) \,\Big], \qquad r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

A *single* ratio per step: no product, so no exponential variance in T.

Solving problem 1: one ratio per step instead of a product

Reuse a batch collected with $\pi_{\theta_{\text{old}}}$, but reweight *one transition at a time* instead of whole trajectories:

$$\nabla_\theta J(\theta) \;\approx\; \mathbb{E}_{s_t \sim \rho_{\theta_{\text{old}}},\, a_t \sim \pi_{\theta_{\text{old}}}(\cdot|s_t)} \Big[\, r_t(\theta)\, G_t\, \nabla_\theta \log \pi_\theta(a_t|s_t) \,\Big], \qquad r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

A *single* ratio per step: no product, so no exponential variance in T.

Ojo: This is an approximation ($\approx$)

The *action* correction $r_t(\theta)$ is exact. The *state* distribution is not corrected: the states in the batch were visited by $\pi_{\theta_{\text{old}}}$, and we assume $\rho_\theta \approx \rho_{\theta_{\text{old}}}$. This holds only while θ stays **close** to θ_{old} $\Rightarrow$ the clipping of slide 27 is what enforces the assumption.

Solving problem 2: give each step its own credit

The trajectory return $G(\tau)$ sits *outside* the sum over t, so every action of the episode receives the same weight. Replace it by the **reward-to-go**:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\Big[\sum_t G_t \nabla_\theta \log \pi_\theta(a_t|s_t)\Big], \qquad G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k$$

Solving problem 2: give each step its own credit

The trajectory return $G(\tau)$ sits *outside* the sum over t, so every action of the episode receives the same weight. Replace it by the **reward-to-go**:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\Big[\sum_t G_t \nabla_\theta \log \pi_\theta(a_t|s_t)\Big], \qquad G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k$$

Exact, not an approximation

Rewards obtained *before* t cannot be caused by a_t; in expectation they contribute zero to the gradient (causality, slide 11). Dropping them leaves the gradient unchanged and removes variance.

Solving problem 2: give each step its own credit

The trajectory return $G(\tau)$ sits *outside* the sum over t, so every action of the episode receives the same weight. Replace it by the **reward-to-go**:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\Big[\sum_t G_t \nabla_\theta \log \pi_\theta(a_t|s_t)\Big], \qquad G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k$$

Exact, not an approximation

Rewards obtained *before* t cannot be caused by a_t; in expectation they contribute zero to the gradient (causality, slide 11). Dropping them leaves the gradient unchanged and removes variance.

Now each step carries its own weight G_t: in the 100-step example, $t=3$ and $t=7$ are no longer forced to move together.

One more variance reduction: the baseline

Motivation:

- If all returns are positive (say $G_t \in [100, 102]$), every sampled action gets its log-probability pushed *up*; the policy improves only through tiny relative differences.
- Subtracting the return we typically expect at s_t rescales this to $-1, 0, +1$: better than usual $\Rightarrow$ up, worse than usual $\Rightarrow$ down.

One more variance reduction: the baseline

Motivation:

- If all returns are positive (say $G_t \in [100, 102]$), every sampled action gets its log-probability pushed *up*; the policy improves only through tiny relative differences.
- Subtracting the return we typically expect at s_t rescales this to $-1, 0, +1$: better than usual $\Rightarrow$ up, worse than usual $\Rightarrow$ down.

Solution:

- Subtract a reference level $b(s_t)$ that depends on the state only. This define the **advantage**: $A_t = G_t - b(s_t)$.
- Usually $b(s_t) = V(s_t)$, then $A_t = G_t - V(s_t)$ indicates how much better was a_t than what we expect in s_t

One more variance reduction: the baseline

Motivation:

- If all returns are positive (say $G_t \in [100, 102]$), every sampled action gets its log-probability pushed *up*; the policy improves only through tiny relative differences.
- Subtracting the return we typically expect at s_t rescales this to $-1, 0, +1$: better than usual $\Rightarrow$ up, worse than usual $\Rightarrow$ down.

Solution:

- Subtract a reference level $b(s_t)$ that depends on the state only. This define the **advantage**: $A_t = G_t - b(s_t)$.
- Usually $b(s_t) = V(s_t)$, then $A_t = G_t - V(s_t)$ indicates how much better was a_t than what we expect in s_t

The estimator we will actually use

$$\nabla_\theta J(\theta) \approx \mathbb{E}_{s_t \sim \rho_{\theta_{\text{old}}},\, a_t \sim \pi_{\theta_{\text{old}}}(\cdot|s_t)} \left[r_t(\theta)\, A_t\, \nabla_\theta \log \pi_\theta(a_t|s_t) \right]$$

Next slide: why replacing G_t by A_t leaves the gradient unchanged.

Why we are allowed to subtract $b(s_t)$

The extra term has zero mean. For a fixed state s:

$$\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}\left[\nabla_\theta \log \pi_\theta(a|s)\, b(s)\right] = \sum_a \pi_\theta(a|s)\, \frac{\nabla_\theta \pi_\theta(a|s)}{\pi_\theta(a|s)}\, b(s) = b(s)\, \nabla_\theta \underbrace{\sum_a \pi_\theta(a|s)}_{=1} = 0$$

The zero comes from normalization (probabilities sum to one for every s), not from any property of b. It is the score-function identity $\mathbb{E}[\nabla_\theta \log p_\theta] = 0$. In continuous action spaces replace $\sum_a$ by $\int_{\mathcal{A}} da$: the argument is identical.

Why we are allowed to subtract $b(s_t)$

The extra term has zero mean. For a fixed state s:

$$\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}\left[\nabla_\theta \log \pi_\theta(a|s)\, b(s)\right] = \sum_a \pi_\theta(a|s)\, \frac{\nabla_\theta \pi_\theta(a|s)}{\pi_\theta(a|s)}\, b(s) = b(s)\, \nabla_\theta \underbrace{\sum_a \pi_\theta(a|s)}_{=1} = 0$$

The zero comes from normalization (probabilities sum to one for every s), not from any property of b. It is the score-function identity $\mathbb{E}[\nabla_\theta \log p_\theta] = 0$. In continuous action spaces replace $\sum_a$ by $\int_{\mathcal{A}} da$: the argument is identical.

Fine print

- b must not depend on a, otherwise it cannot leave the sum over actions.
- b must not depend on θ; in code, `returns - values.detach()`.
- Zero *in expectation*, not per sample: $G_t\, \nabla_\theta \log \pi_\theta \neq A_t\, \nabla_\theta \log \pi_\theta$ on any single trajectory. Same mean, smaller variance — that difference is the point.
- Holds for each fixed s, so it survives the state-distribution mismatch of the previous slide (with $a \sim \pi_{\theta_{\text{old}}}$, the ratio gives $b(s)\nabla_\theta \sum_a \pi_\theta(a|s) = 0$ just the same).

From a gradient to a loss we can optimize

We have a gradient estimator, but PyTorch needs a *scalar* to differentiate. Treat A_t as a constant (computed from the collected batch, no gradient flows through it). Then:

$$\nabla_\theta \left[r_t(\theta)\, A_t \right] \;=\; A_t \,\frac{\nabla_\theta \pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \;=\; r_t(\theta)\, A_t \,\nabla_\theta \log \pi_\theta(a_t|s_t)$$

which is exactly the estimator of slide **??**.

Surrogate objective

$$L^{\text{IS}}(\theta) = \mathbb{E}_t \left[r_t(\theta)\, A_t \right]$$

From a gradient to a loss we can optimize

We have a gradient estimator, but PyTorch needs a *scalar* to differentiate. Treat A_t as a constant (computed from the collected batch, no gradient flows through it). Then:

$$\nabla_\theta \left[r_t(\theta)\, A_t \right] \;=\; A_t \,\frac{\nabla_\theta \pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \;=\; r_t(\theta)\, A_t \,\nabla_\theta \log \pi_\theta(a_t|s_t)$$

which is exactly the estimator of slide **??**.

Surrogate objective

$$L^{\text{IS}}(\theta) = \mathbb{E}_t \left[r_t(\theta)\, A_t \right]$$

Sanity check. At $\theta = \theta_{\text{old}}$ we get $r_t = 1$ and $\nabla_\theta L^{\text{IS}} = \mathbb{E}_t[A_t \nabla_\theta \log \pi_\theta]$: we recover REINFORCE with baseline. The first gradient step is identical; importance sampling only matters from the second step onwards.

From a gradient to a loss we can optimize

We have a gradient estimator, but PyTorch needs a *scalar* to differentiate. Treat A_t as a constant (computed from the collected batch, no gradient flows through it). Then:

$$\nabla_\theta \left[r_t(\theta)\, A_t \right] \;=\; A_t \,\frac{\nabla_\theta \pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \;=\; r_t(\theta)\, A_t \,\nabla_\theta \log \pi_\theta(a_t|s_t)$$

which is exactly the estimator of slide **??**.

Surrogate objective

$$L^{\text{IS}}(\theta) = \mathbb{E}_t \left[r_t(\theta)\, A_t \right]$$

Sanity check. At $\theta = \theta_{\text{old}}$ we get $r_t = 1$ and $\nabla_\theta L^{\text{IS}} = \mathbb{E}_t[A_t \nabla_\theta \log \pi_\theta]$: we recover REINFORCE with baseline. The first gradient step is identical; importance sampling only matters from the second step onwards.

Why this is the key step

$\nabla_\theta J$ can only be evaluated *at* the policy that generated the data. $L^{\text{IS}}(\theta)$ can be evaluated and differentiated for $\theta \neq \theta_{\text{old}} \;\Rightarrow$ we can take K gradient epochs on the same batch (slide **??**).

$V(s_t)$ can take different forms, used as an estimation of the value function leads to Proximal Policy Optimization or PPO (Schulman et al., 2017).

$V(s_t)$ as the value function (Actor-Critic scheme)

Recall

$$V_\theta^\pi(s_t) = \mathbb{E}_{\pi_\theta}\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \;\middle|\; s_t\right]$$

Why use it? The advantage $A_t = G_t - V(s_t)$ answers:

> *"Was action a_t better or worse than what we normally expect in s_t?"*

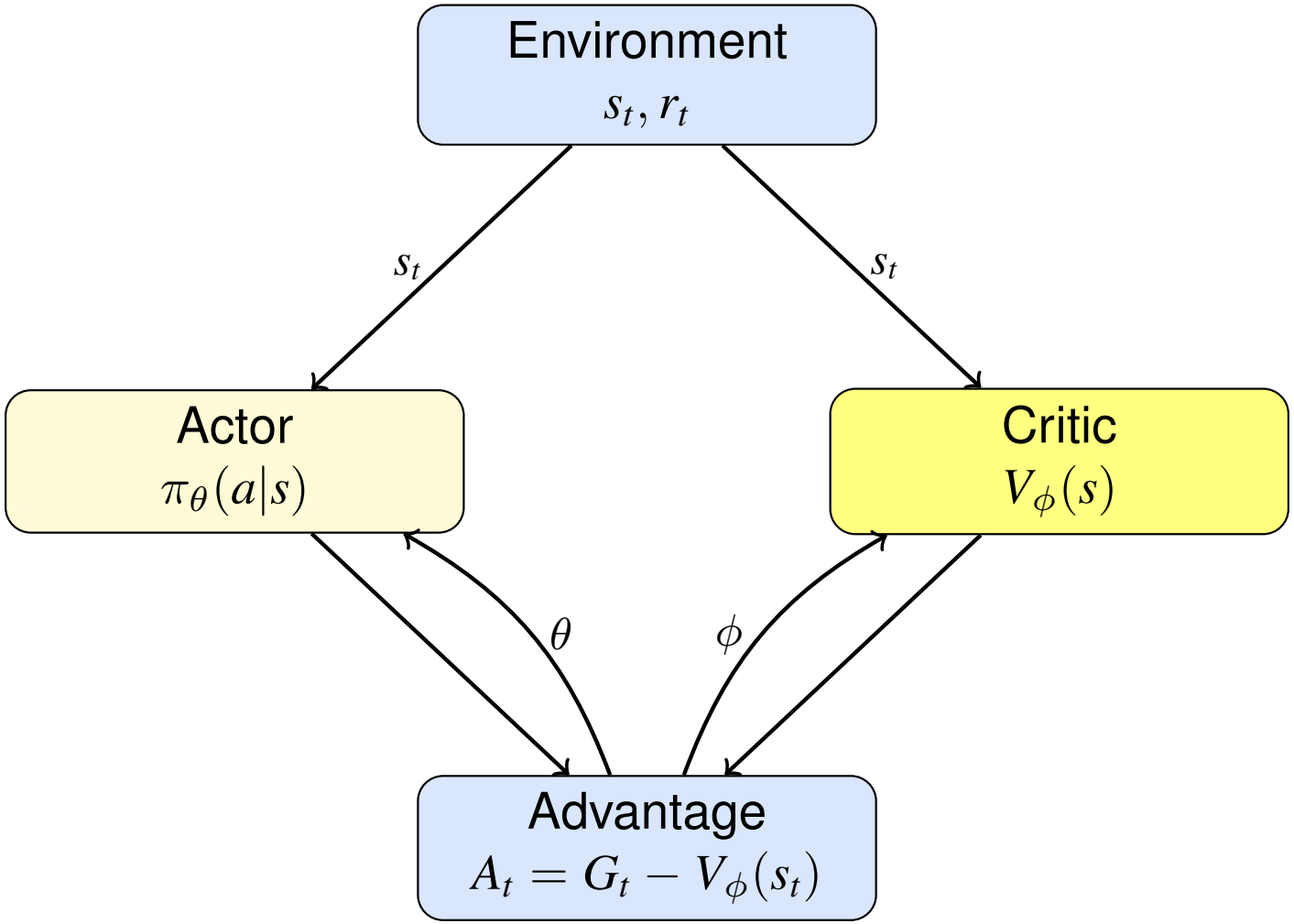
How to compute $V(s_t)$: Actor-Critic

Train a second network $V_\phi(s_t)$ the **critic** to predict the expected return from each state:

$$\mathcal{L}_{\text{critic}} = \mathbb{E}_t\left[(V_\phi(s_t) - G_t)^2\right]$$

The **actor** π_θ uses V_ϕ to compute advantages and update the policy.

Critic learns which states are inherently good or bad. Actor needs to favor actions that are better than expected.

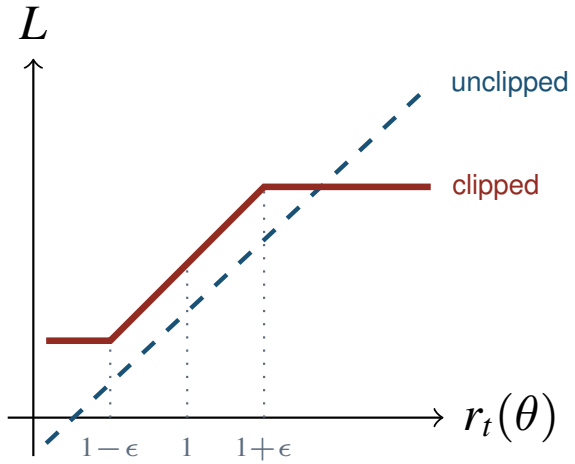

Environment
s_t, r_t
s_t
s_t
Actor
$\pi_\theta(a|s)$
Critic
$V_\phi(s)$
θ
ϕ
Advantage
$A_t = G_t - V_\phi(s_t)$

The Problem with Unconstrained Updates

$L^{\text{IS}}(\theta)$ allows reusing data but introduces a new danger:

If θ moves *too far* from θ_{old}:

- The ratio $r_t(\theta)$ becomes very large or very small
- Importance sampling estimate is no longer reliable
- Gradient steps become destructively large



Key insight (PPO)

We want to take the **largest possible improvement step** without going so far that the policy becomes unreliable.

PPO: Proximal Policy Optimization

Core idea: clip the probability ratio so updates stay close to $\pi_{\theta_{\text{old}}}$.

PPO-Clip Objective (Schulman et al., 2017)

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t[\min\{r_t(\theta)\,A_t,\ \text{clip}(r_t(\theta),\ 1-\epsilon,\ 1+\epsilon)\,A_t\}]$$

where $r_t(\theta) = \dfrac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ and $\epsilon \approx 0.2$.

How the clip works:

- If $A_t > 0$ (good action): ratio is clipped at $1+\epsilon$ can only increase probability so much
- If $A_t < 0$ (bad action): ratio is clipped at $1-\epsilon$ can only decrease probability so much
- Takes the **pessimistic** (min) bound conservative by design

PPO Algorithm

Algorithm 2 PPO-Clip (Schulman et al., 2017)

Initialize policy π_θ and value network V_ϕ
repeat
    Collect set of trajectories $\mathcal{D}$ under current π_θ
    Compute advantages $\hat{A}_t$
    for K epochs **do**
        Sample mini-batches from $\mathcal{D}$
        Compute $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$
        Update θ by maximizing $L^{\text{CLIP}}(\theta)$
        Update ϕ by minimizing value loss $(V_\phi(s_t) - G_t)^2$
    end for
    $\theta_{\text{old}} \leftarrow \theta$
until converged

Key difference from REINFORCE: data $\mathcal{D}$ is reused for K gradient epochs before being discarded

REINFORCE vs PPO

Property	**REINFORCE**	**PPO**
Data reuse	None (on-policy)	K epochs per batch
Variance	High	Lower
Update stability	Unconstrained	Clipped ratio
Value network	No	Yes (V_ϕ)
Sample efficiency	Low	Moderate
Implementation	Simple	Moderate

Why PPO dominates in practice

PPO keeps the **simplicity of policy gradient** (no Q-function, no replay buffer) while fixing the two main failure modes of REINFORCE: high variance and unstable updates.